Project Phase 1 Report

for

Software Architecture and Design Project
KWIC Index System and Search Engine

Version 1.0
October 15, 2026

Course Information
Course: SE 6362.001
Instructor: Dr. Lawrence Chung

Team Website
https://kalani-kawaguchi.github.io/SE6362-SWArch-Project/

Team Profile

Member	Role	Contributions
Jonathan Loper	Team Leader System Architect	Meeting Coordination Architectural Design KWIC Module Implementations Website Layout
Kalani Kawaguchi	Backend Specialist	Backend Setup KWIC Module Revision Frontend/Backend Integration and Deployment
Zhi Li	UI/UX Designer	User Interface Design Styling
Mitchell Vu	Frontend Developer	Requirements Frontend Setup System Documentation User Manual

Meeting History

Meeting Date	Agenda	Attendees
August 28 @ 7:00pm	- Discuss preliminary project plan - Choose project leader	Jonathan Loper Kalani Kawaguchi Mitchell Vu
September 10 @ 6:45pm	- Choose technologies and frameworks - Set up GitHub with boilerplate React and Spring Boot applications - Discuss preliminary requirements and architectural decisions	Jonathan Loper Kalani Kawaguchi Zhi Li Mitchell Vu
September 18 @ 6:00pm	- Finalize architectural design. - Implementation of KWIC Modules.	Jonathan Loper Kalani Kawaguchi Mitchell Vu
September 29 @ 7:00pm	- Frontend/Backend integration - Plan and practice presentation	Jonathan Loper Kalani Kawaguchi Zhi Li Mitchell Vu

Contents

1. Introduction 1
2. Requirements Specification 2
    2.1 Functional Requirements: 2
    2.2 Non-functional Requirements: 3
3. Architectural Specification 4
    3.1 Style: Object Oriented 4
    3.2 Components: Objects 4
    3.3 Connections: Method Invocation 4
    3.4 Constraints 5
    3.5 Pattern: Adapter/Wrapper 5
    3.6 Rationale 6
        Advantages 6
        Disadvantages 6
    3.7 Requirements Traceability 7
    3.8 Architectural Diagram 8
    3.9 Class Diagram 9
4. Prototype Implementation 10
    4.1 Technology Stack 10
    4.2 Deployment Services 10
    4.3 File Structure/Descriptions 10
    4.4 Sample Input/Output 11
    4.5 Test Plan 14
5. User Manual 15
    5.1 Navigation 15
    5.2 KWIC Index Builder 15
    5.3 Index Search 16
6. Glossary 17
7. References 17
8. Comparative Evaluation of Project Deliverable 18

1. Introduction

A Keyword-in-Context (KWIC) index system organizes information by indexing keywords along with the surrounding text in which they appear. It allows users to quickly find relevant information by searching for specific keywords and viewing their context. In our project, we will use a KWIC index as a component of our search engine to help organize and retrieve relevant website information based on user search queries.

This phase of the project consists of two applets, the KWIC Index Builder, and the prototype Search Engine.

- The KWIC Index Builder allows users to enter lines of text and generate their own KWIC index. This user generated index is not saved to the database, as it is intended for testing and demo purposes.
- The search engine allows users to enter keywords and returns web results with descriptions containing those keywords. Searches are conducted by querying a database containing a KWIC index generated by our KWIC system, and a list of corresponding webpages. Currently, our prototype only contains data from 100 web pages. As a result, many searches will not return any results. Our plan is to expand the search engine's capabilities in phase 2 of this project.

2. Requirements Specification

2.1 Functional Requirements:

REQ-ID	DESCRIPTION
FR-001	The KWIC (Key Word in Context) index system shall accept a file containing an ordered set of lines, where each line is an ordered set of words, and each word is an ordered set of characters.
FR-002	Lines read from the file shall be filtered from punctuation marks and other non-alphanumeric symbols before processing. In addition, blank lines shall be discarded.
FR-003	Each line shall optionally be associated with a web address (URL), in which case, the line shall represent a description of the addressed website.
FR-004	Each line shall be “circularly shifted” by repeatedly removing the first word and appending it at the end of the line.
FR-005	The system shall store all circular shifts of every input line in ascending alphabetical order, ignoring case. The stored lines make up the KWIC index.
FR-006	Duplicate lines shall be removed if their respective URLs match, or if they are not associated with URLs. (For example, the line “xyz xyz” has two identical shifts, so only one shall be stored).
FR-007	The KWIC system shall be utilized in two applets, a KWIC Index Builder, and a Search Engine.
FR-008	The graphical user interface (GUI) for the KWIC Index Builder shall enable a user to provide keyboard input and generate a KWIC index by pressing a button.
FR-009	The KWIC Index Builder GUI shall display the circular shifts and alphabetization of each input line separately, as well as the cumulative alphabetization of all circular shifts of all lines.
FR-010	The KWIC Index Builder GUI shall provide a button to clear all input, which shall delete all stored lines and clear the display.
FR-011	The Search Engine applet shall create its own KWIC index using a database of web URLs and associated descriptions. The descriptions shall be used to build the index.
FR-011	The Search Engine shall accept a set of search words and return the URLs associated with all KWIC index entries containing any of those words.
FR-012	The Search Engine GUI shall provide a search bar where users can type search words and a button to run the search on those words and display the results.

2.2 Non-functional Requirements:

REQ-ID	DESCRIPTION
NFR-001	Understandability: developers shall be able to understand system architecture and correctly predict the results of API calls at a glance.
NFR-002	Portability: the system shall be compatible with both x86 and ARM hardware and shall be able to run in both Windows and UNIX environments.
NFR-003	Enhanceability: The system shall support the addition of new components and intermediate processing steps without making modifications to existing ones.
NFR-004	Reusability: Each step in the index generation process shall be reusable in other systems or contexts without modification.
NFR-005	Performance: KWIC index lookup time shall not exceed 2 seconds.
NFR-006	User Friendly: The graphical user interface shall be self-explanatory, including detailed labels, placeholders, and error messages, so the user can learn to operate the system correctly without reading a manual.
NFR-007	Responsiveness: The user interface shall remain responsive while index generation and searching is taking place. Proper messages shall inform the user when processing begins, is completed, or is halted for any reason.
NFR-008	Adaptability: The system components shall be able to utilize different data structures and processing algorithms interchangeably, without interfering with other components.

3. Architectural Specification

3.1 Style: Object Oriented

3.2 Components: Objects

- Master Control: coordinates operations between components
- Input Buffer: stores raw input received until it can be processed
- Line Storage: parses and store input lines before processing
- Circular Shift: generates all circular shifts of input lines
- Alphabetizer: sorts lines in alphabetical order
- Merged Lines: merges current index with newly processed lines
- Output Buffer: stores output lines waiting to be copied to database

3.3 Connections: Method Invocation

During program execution, objects communicate via method invocation in the following order:

- **runIncremental()** in Master Control calls **readFromFile()** in Input Buffer, which creates a new thread to read the input file line-by-line. Master Control then creates multiple threads which each perform the remaining operations as input is received.
- **runIncremental()** calls **getInputLine()** in Input Buffer to get the next line of input.
    - **GetInputLine()** calls **storeLine()** in Line Storage to parse and store the next line.
- **runIncremental()** calls **circularShiftLines()** in Circular Shift to generate shifts.
    - **circularShiftLines()** calls **getWordCount()** and **getWord()** to read data from Line Storage. It then generates the shifts.
- **runIncremental()** calls **alphabetizeLines()** in Alphabetizer to sort the lines alphabetically.
    - **aphabetizeLines()** calls **getWordCount()** and **getWord()** to read data from the Circular Shift. It then sorts the lines alphabetically.
- **runIncremental()** calls **setOutputLines()** in Output Buffer to store the sorted results.
    - **setOutputLines()** calls **mergeLines()** in Merged Lines to merge preexisting output with the newly generated output.
        - **mergeLines()** calls **getWordCount()** and **getWord()** to read data from the Alphabetizer. It then merges this data with any data currently stored in the Merged Lines module.
- Once processing is complete, **saveOutput()** in Master Control calls **writeToDatabase()** in Output buffer to move all data stored in the buffer to the database.
    - **writeToDatabase()** calls **getWordCount()** and **getWord()** to read data from the Merged Lines. It then inserts all data into the database.

3.4 Constraints

- Objects do not share data; data must be copied from one object to another
- Objects can only communicate using method invocation.
- Objects can only call the public methods of other objects. Private and protected methods can only be called from within the object. (Protected status is the same as private, except protected methods in an abstract class can be called from the concrete class.)
- Only private and protected methods can directly modify the data structure containing the text lines. For example, setWord() in Alphabetizer is protected since it directly modifies the data. Thus, the Master Control cannot modify data in Alphabetizer using setWord(), but must instead use the alphabetizeLines() public method.
- Line Storage, Circular Shift, Alphabetizer, and Merged Lines share a uniform interface for getting and setting words.
- Only the Master Control is visible to the “outside world”.

3.5 Pattern: Adapter/Wrapper

- Line List Class and Line Class serve as wrappers for string arrays, providing a clean interface for users to access individual words without directly accessing the array.

3.6 Rationale

Advantages

- Details of data representation and algorithms are wrapped in layers of abstraction, resulting in better modifiability, enhanceability, and adaptability
    - Changes to data representation in one module do not affect the data representations in other modules. Contrast with shared data style, where data representation changes would impact every module.
    - Changes to processing algorithms in one module also does not affect other modules. For example, changing from incremental to batch processing would only affect Master Control (i.e. runIncremental would be replaced with runBatch). Contrast with shared data style, where each module would need to have its data access subroutines modified to process in batch.
    - Adding a new module (for example, noise word eliminator) would not necessitate modifications to other modules, and the new module could inherit the same uniform interface as the others. Contrast with shared data style, where a new module might create new global data that other modules would need to interact with.
- Tight cohesion within components makes them easier to understand and maintain. Contrast with shared data style, where it is difficult to understand a component without knowledge of the components that came before it, since they operate on the same data.
- Few assumptions about component interactions make them very reusable usually. Contrast with shared data style, where a component is only usable in the context of the global data it shares with other components.

Disadvantages

- Copying data from one object to another incurs additional time and memory costs. Contrast with shared data style, where all modules can access the same data without copying it.
- Communication via method invocation necessitates coupling, reducing maintainability. For example, if we wanted to reuse a module from another system to perform a processing step between Circular Shift and Alphabetizer, the new module would need to have its interfaces modified so that it could read data from the Circular Shift module and be read by the Alphabetizer. Contrast with pipe and filter, where any module that accepts input and produces output can be inserted into the pipe and filter chain without modification. Although the object-oriented style has worse coupling than the pipe and filter style, it is still better than the shared data style, where all components are accessing the same global data.

3.7 Requirements Traceability

Each nonfunctional requirement is scored based on its prioritization in the architecture:

5 pts. = Excellent

4 pts. = Good

3 pts. = Average

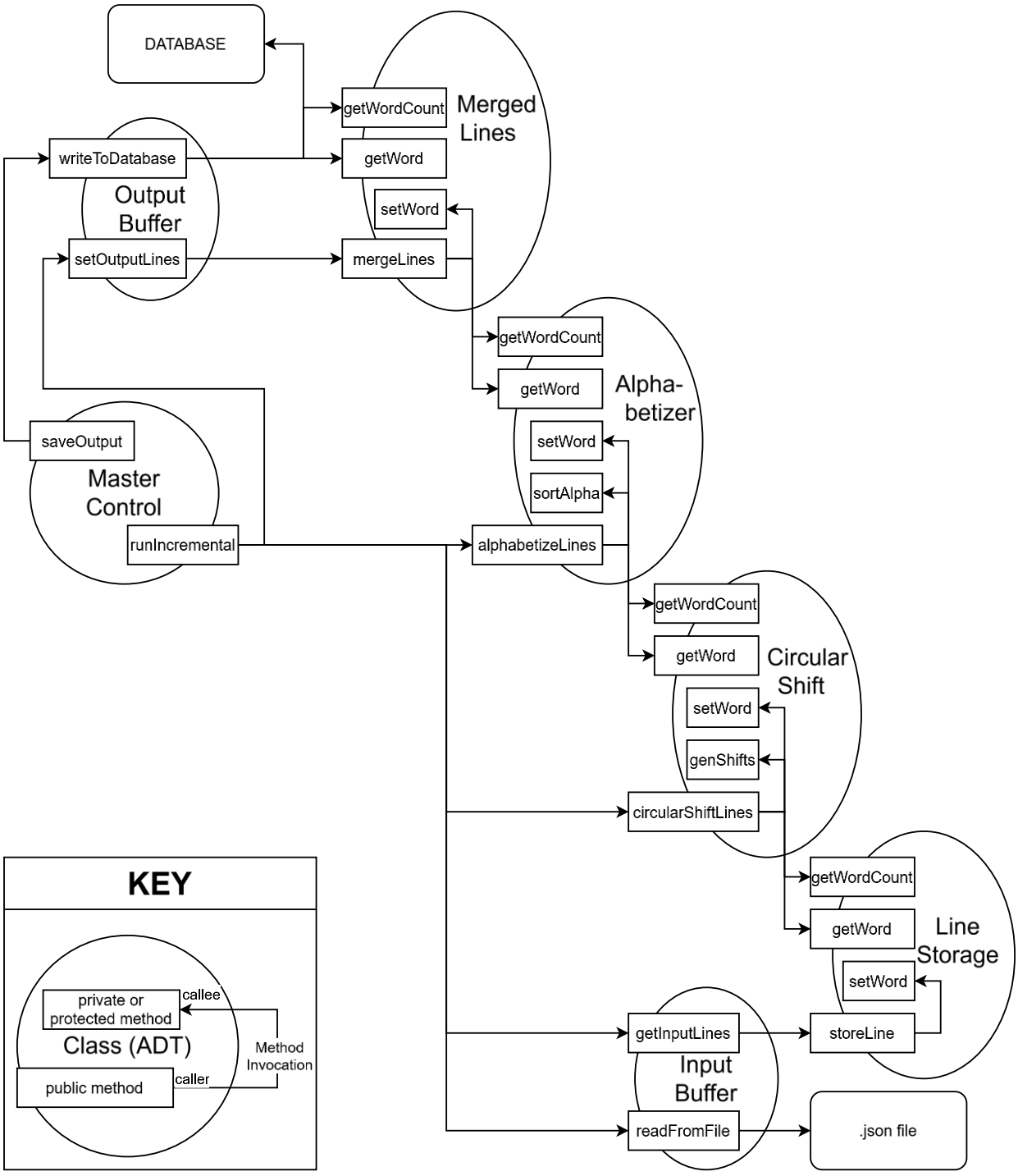
2 pts. = Acceptable

1 pt. = Poor

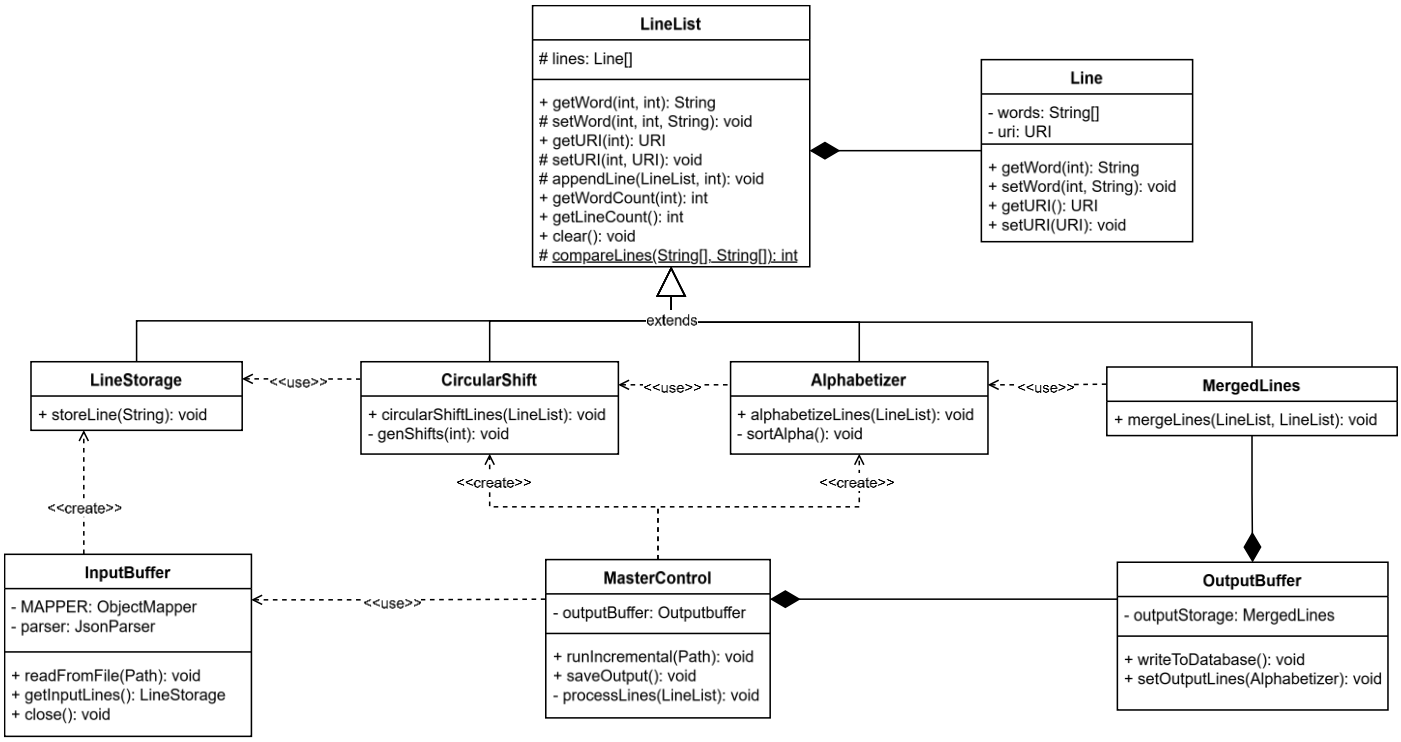
Nonfunctional Requirement	Score	Reason
Understandability	5	The architectural diagram clearly shows caller and callee in each method invocation relationship, which makes it easy to see how components interact.
Enhanceability	4	Adding new components is usually easy as long as the new component can seamlessly interact with existing components via their public method interfaces.
Reusability	4	While components must be aware of each other to communicate, they make relatively few assumptions about such interactions, which makes them fairly reusable.
Memory Usage	2	Copying data from one object to another can greatly increase memory utilization.
Response Time	3	Copying data also affects response time, but incremental and concurrent processing help mitigate the impact of this.
Adaptability	5	Data structures and processing algorithms are only accessed by private and protected methods, so they can be changed easily without modifying higher level interactions.

Note: Portability, User-Friendliness, and Responsiveness are not mentioned here since these NFRs are not directly related to KWIC system architecture (portability depends on the chosen language and runtime environment; user-friendliness and responsiveness are related to the user interface design, which is not part of this architectural specification).

3.8 Architectural Diagram



3.9 Class Diagram



4. Prototype Implementation

4.1 Technology Stack

- Frontend: React (JavaScript)
- Backend: Spring Boot (Java)
- Database: MySQL Database

4.2 Deployment Services

- Frontend: GitHub Pages
- Backend: Railway

4.3 File Structure/Descriptions

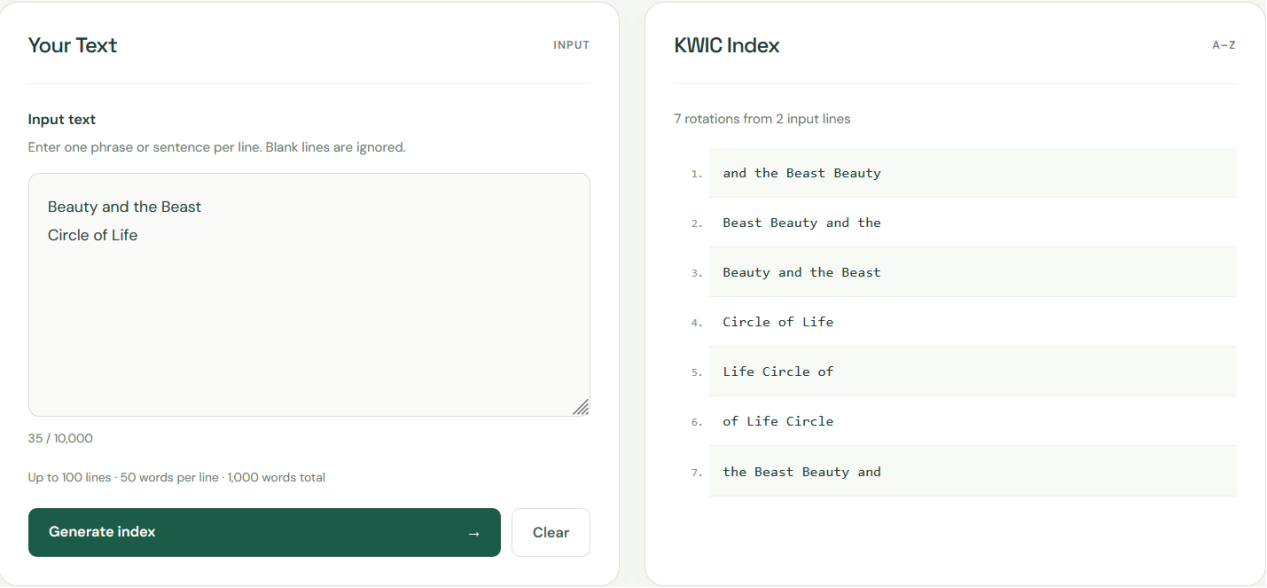
Following is a stripped-down representation of our file directories showing the locations of all of our source files. All files are included with the final submission.

- Backend
    - src
        - main\java\com\se6362\kwic
            - controller ← REST controllers for each applet
                - HealthController.java
                - KwicController.java
                - SearchController.java
            - core ← KWIC system modules are here
                - Alphabetizer.java
                - CircularShift.java
                - InputBuffer.java
                - Line.java
                - LineList.java
                - MasterControl.java
                - MergedLines.java
                - OutputBuffer.java
            - database ← database communication
                - IndexDatabase.java
            - BackendApplication.java ← entry point for Java backend
        - test\java\com\se6362\kwic
            - KwicSystemTest.java ← system test for KWIC modules
            - LineListTest.java ← unit tests for individual modules

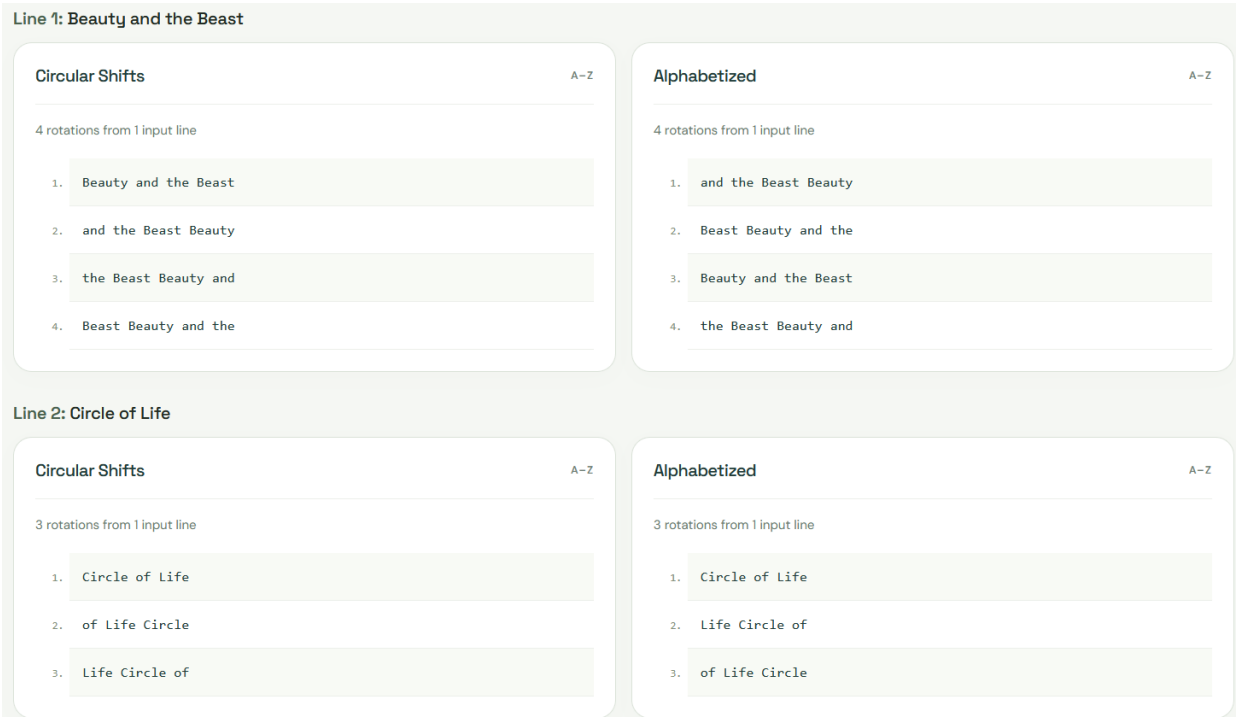
- frontend
  - src
    - pages ← react components for each page
      - Home.jsx
      - KWIC.jsx
      - Search.jsx
    - App.jsx ← root react component
    - KwicResultRow.jsx ← incremental results component
    - ResultPanel.jsx ← cumulative results component
    - Searchresult.jsx ← search results component
    - styles.css ← styles for all components

4.4 Sample Input/Output

KWIC Input (left side) and Cumulative Output (right side)



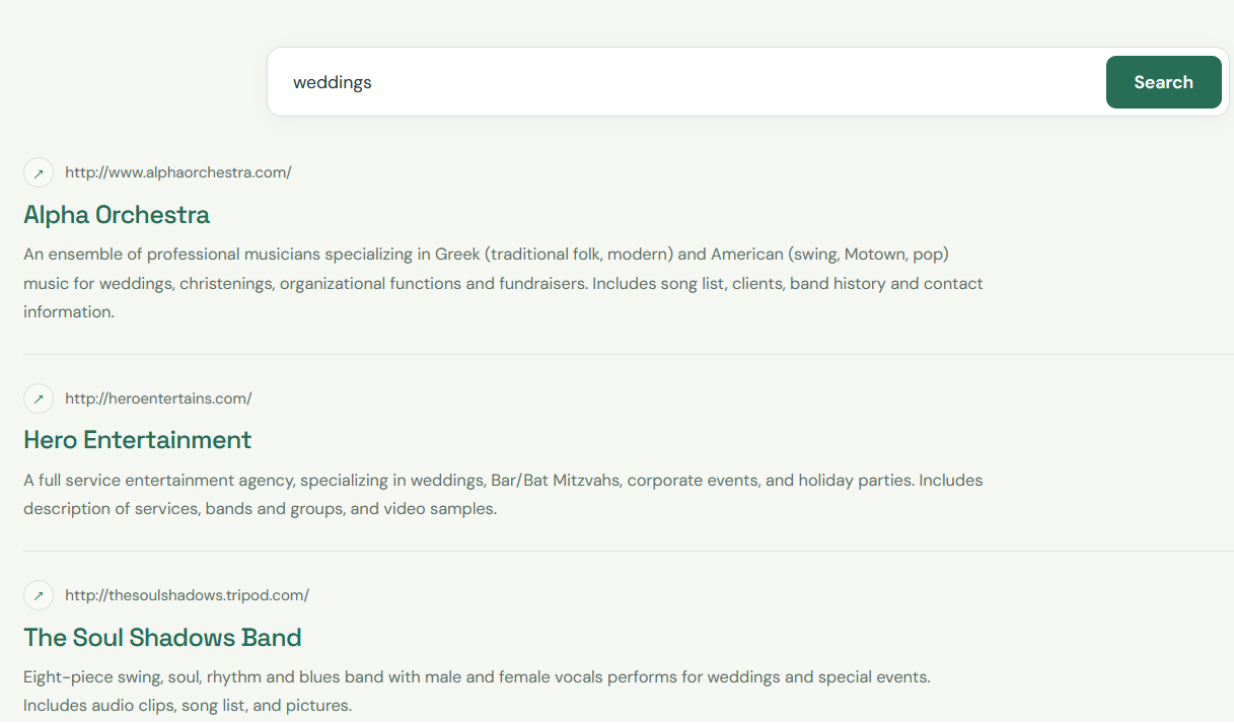
Incremental Output for Above Input:



Sample KWIC Index Database Entries for Search Engine

Keyword	ShiftText	PageTitle
visa	visa health passport currency and customs re...	World Travel Guide - Algeria
visit	visit requests Planned community developer In...	Crown Asia
vocals	vocals performs for weddings and special eve...	The Soul Shadows Band
War	War of 1812 Complete online edition of Lossing...	Lossing's Field Book of the War of 1812
warehouse	warehouse with large selections of low to high...	Melrose Carpets
warranties	warranties location hours of operation and eve...	Chestatee Ford
Washington	Washington after the US attacks PJ Harvey wh...	Award Winner Caught in US Chaos
web	web design ecommerce and brand identity ser...	Purple, Rock, Scissors
weddings	weddings and special events Includes audio cl...	The Soul Shadows Band
weddings	weddings BarBat Mitzvahs corporate events a...	Hero Entertainment

Example Search Results for Keyword “weddings”



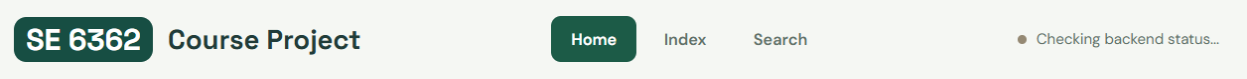
4.5 Test Plan

- Unit Test for the KWIC modules can be found in LineListTest.java. These tests show that the Line Storage, Circular Shift, Alphabetizer, and Merged Lines modules produce the correct output independently of each other.
- The KwicSystemTest.java file contains a test of the whole KWIC process from beginning to end, starting with file input, and incrementally producing an alphabetized index.
- Integration tests for the KWIC Index Builder involve testing the frontend/backend interaction by entering various inputs and predicting the output. Following are examples of such tests I have performed.
    - Entering a single line of input with n unique words into the KWIC Index Builder should produce n lines of output.
    - Entering m lines of input with n unique words each should produce n * m lines of output.
    - Entering a line of blank spaces should produce no output.
    - Entering a line containing all punctuation should produce no output.
    - Entering “A!!!” and “!!!B” as input should produce output lines “A B” and “B A”.
    - Entering “A+B+C” should produce output “ABC”.
    - Entering “a, b” and “c, d” should produce “a b”, “b a”, “c d”, “d c”.
    - Entering “a a a a” should produce only “a a a a” since all circular shifts are duplicates.
    - Entering “a” and “a” should produce only “a” since they are also duplicates.
- Further Search Engine testing will be conducted in phase 2.

5. User Manual

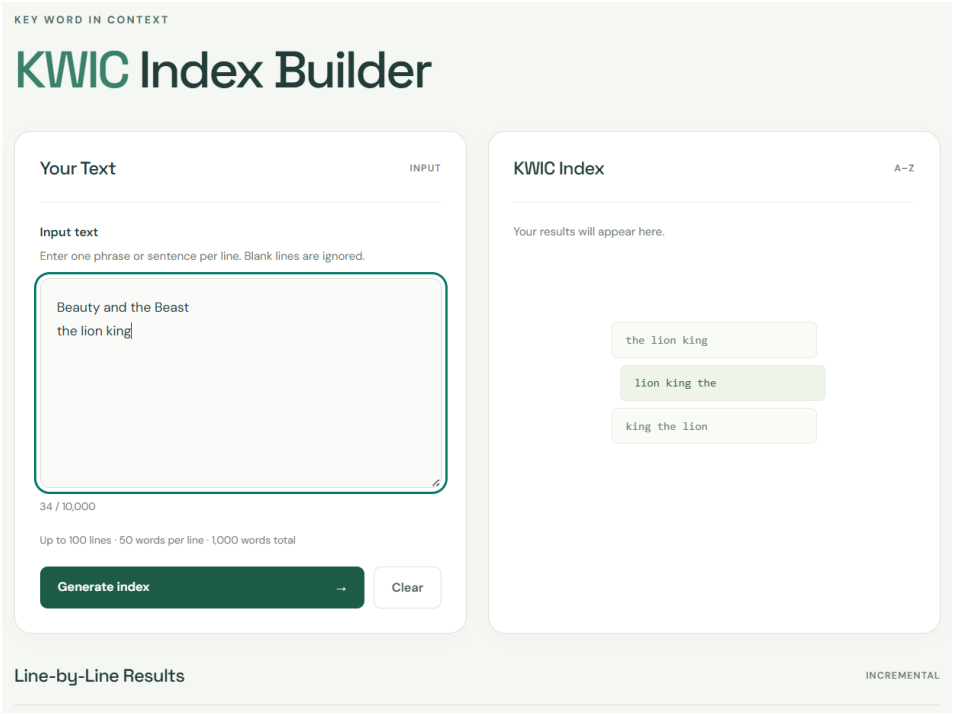
Working prototype available at https://kalani-kawaguchi.github.io/SE6362-SWArch-Project/

5.1 Navigation



Our KWIC system contains 2 tools: a KWIC Index Builder and a Web Search Engine. These tools can be accessed by clicking the corresponding buttons on the top menu bar.

5.2 KWIC Index Builder

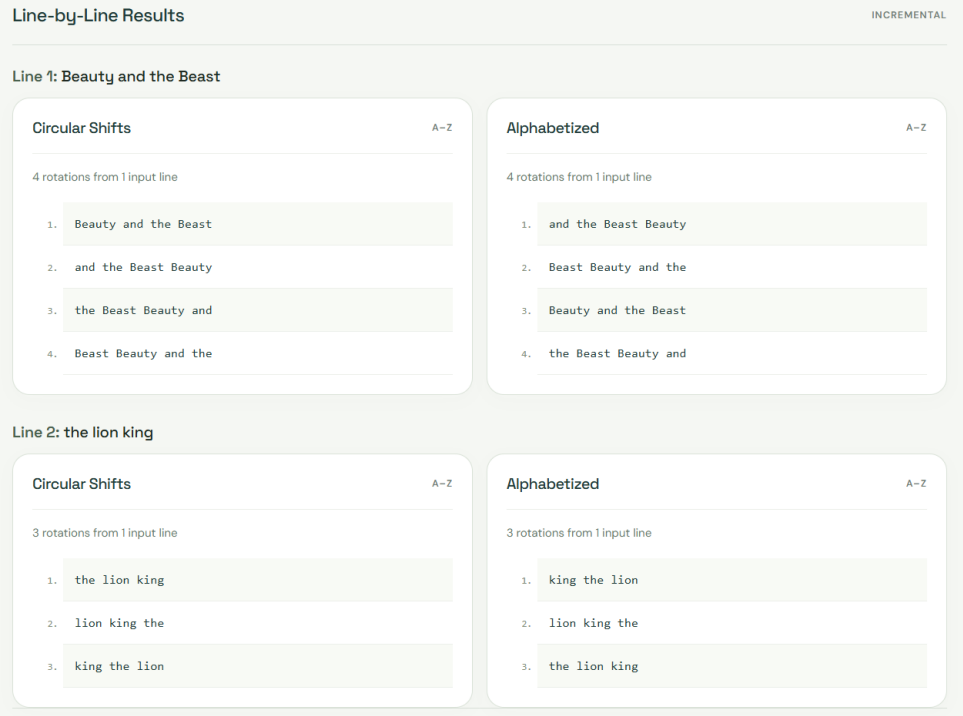


The KWIC Index Builder will generate circular shifts and then save them to the index table based on user input.

Enter the lines into the text area on the left. Separate each line by creating a new line using the Enter key. Each entry supports up to 100 lines with up to 50 words per line and up to 1,000 words in total.

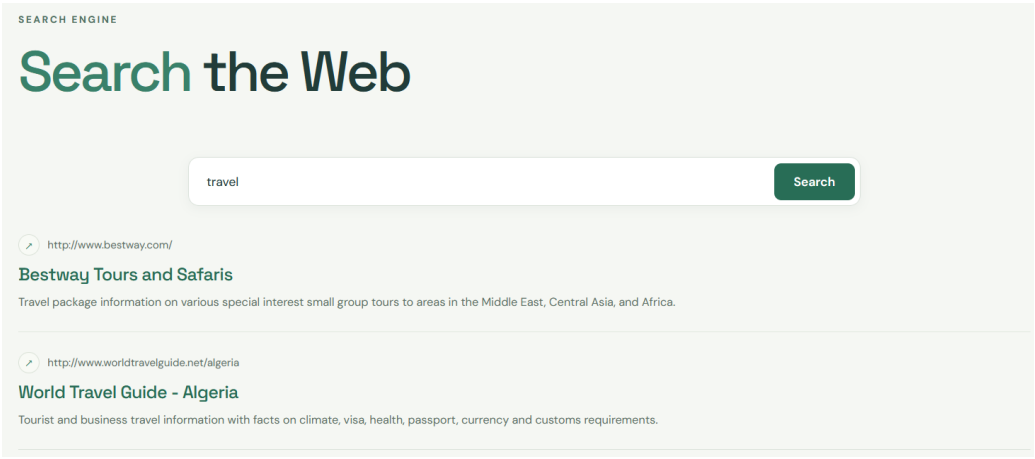
Click the “Generate index” button. Your index will appear in the “KWIC Index” box on the right. Note that pressing “Generate index” again does not erase any index you previously created but rather adds to it.

Click the “Clear” button to remove all entries from your current index.



Scroll down to see the incremental (line-by-line) results of your index creation.

5.3 Index Search



The search engine will return all results containing the words you enter into the search bar.

Enter any number of keywords into the search bar. (limit 500 characters)

Click “Search” or press enter to view the resulting web pages below.

Click on the name of a webpage displayed to navigate to that page.

6. Glossary

Word: an ordered set of characters

Line: an ordered set of words

Circular Shift: a line obtained by repeatedly removing the first word of a line and appending it at the end of the line. A line with n words has n – 1 possible circular shifts.

Alphabetize: the process of sorting lines in alphabetical order

Merge: the process of taking two alphabetized lines and merging them into a single alphabetized line. The operation takes linear time.

KWIC Index: An index over the set of lines S which contains an alphabetized listing of all circular shifts of each line in S. Searching for an element in the index takes logarithmic time.

7. References

With content from Curlie.org - the largest human-edited directory of the web. Contribute by submitting a website or becoming an editor.

8. Comparative Evaluation of Project Deliverable

Based on our observations of other teams' presentations, we believe that our deliverable is competitive with, and potentially superior to, the other projects presented. The following are my reasons for this assessment:

- Our deliverable emphasizes the value of complete, measurable, and technically achievable requirements. We provide clarification on many functional issues that were ambiguous in the initial requirements provided in the project description, such as how to handle punctuation and blank lines, or where input and output should come from. We also establish measurable ways of achieving non-functional requirements, such as an upper bound on search time for performance, and algorithm interchangeability for adaptability. Most of all, our requirements are prioritized in architectural design, which can be clearly seen from our traceability table.
- Our architectural diagram was the clearest and most understandable of all of those presented. In our diagram, method invocations are denoted using arrows from caller to callee, rather than just from object to callee. Following this convention reveals the underlying structure of chained method invocations, which makes the system much more understandable. Additionally, the objects all follow a uniform interface with getWord(), getWordCount(), and setWord(), which improves both understandability and maintainability while also adhering to the Java naming convention of using get and set for public methods. Moreover, each method performs a single well-defined task, rather than clumping multiple unrelated tasks together. Lastly, the addition of the Merged Lines module provides a much more cohesive structure than using a combined Merged Lines and Output Buffer module.
- Our class diagram provides additional support for our architectural specifications, clearly showing the inheritance and composition relationships between classes.
- Our web implementation with distinct frontend and backend components follows the principle of separation of concerns, separating the business logic from the user interface. In addition, our user interface is clean, intuitive, and understandable, displaying the merged output and the incremental output separately so that the difference between the two is obvious.